

CSCE 627 THEORY OF COMPUTABILITY · MODULE 03 · LECTURE NOTES

Memory as a Stack

Context-free languages, and why parsing works.

MODULE OUTCOMES

- Write a context-free grammar and derive strings from it.
- Explain the equivalence of grammars and pushdown automata.
- Explain ambiguity and why it matters for parsing.
- Prove a language not context-free.
- Place the practical grammar classes in the hierarchy.

1 Grammars and pushdown automata

A CONTEXT-FREE GRAMMAR GENERATES; A PUSHDOWN AUTOMATON RECOGNISES

A grammar is a finite set of rules, each rewriting a single non-terminal into a string of terminals and non-terminals. A string is in the language if some sequence of rewrites starting from the start symbol produces it.

A pushdown automaton is a finite automaton with a stack — it may push, pop, and branch on the symbol currently on top. **The stack is unbounded, and it is the only unbounded memory the machine has,** which is the entire difference from Module 02.

The two formalisms are equivalent, and the proof is an explicit construction in each direction — **which is why 'context-free' is used to describe both a generating device and a recognising one** without ambiguity.

And the stack is exactly what matched counting requires: push a marker for each a, pop one for each b, and accept if the stack is empty at the end — so $\{a^n b^n\}$ is context-free, and Module 02 §2 proved it is not regular. **One bounded counter's worth of unbounded memory, in exactly the shape counting needs.**

Class	Memory discipline	Can express
Regular	Finite state, and nothing else.	Tokens, protocols, validation. Not nesting (Module 02).
Context-free	A stack.	Nesting and matched brackets. Not cross-dependency (§3).

Class	Memory discipline	Can express
Context-sensitive	A tape bounded by the input length.	$a^n b^n c^n$, declare-before-use, type agreement.
Recognisable (Turing-recognisable)	An unbounded tape.	Anything a program can verify when the answer is yes (Module 04).
Decidable	An unbounded tape, plus a guarantee of halting.	Anything a program can decide for every input (Module 05).

The hierarchy is strict at every level — each class properly contains the one above it — and each separation is proved by one of the course's two techniques, which is why Module 02's pumping argument was worth doing carefully. Placing a problem in this hierarchy tells you what machinery it needs and therefore what algorithms are available, which is the practical use of the whole picture.

2 Ambiguity

A grammar is AMBIGUOUS if some string in its language has two distinct parse trees. The classic case:

```
E -> E + E | E * E | id
```

Then "a + b * c" has two parse trees, and THEY MEAN DIFFERENT THINGS. Parsing is not merely recognition: the TREE is the output, and an ambiguous grammar does not determine it.

THE USUAL FIXES

stratify the grammar by precedence level

```
E -> E + T | T ; T -> T * F | F ; F -> id
```

or declare precedence and associativity and let the parser generator resolve the conflicts

-- which is what the yacc family does, and the conflict report is telling you the grammar is ambiguous (CSCE 605 Module 03).

AND SOME LANGUAGES ARE INHERENTLY AMBIGUOUS: no unambiguous grammar exists for them at all. Rare, and it establishes that ambiguity is not always a grammar-writing mistake.

WORSE: WHETHER A GIVEN GRAMMAR IS AMBIGUOUS IS UNDECIDABLE (Module 08). So a parser generator CANNOT tell you that your grammar is unambiguous -- it can only report the conflicts IT found with ITS particular method. Which is why a conflict count is a diagnostic rather than a verdict, and why "zero conflicts" does not mean "unambiguous".

The undecidability of ambiguity is why parser generators report conflicts rather than verdicts — and it is the first concrete payoff in this course of a result we have not yet proved. A practical tool's interface is shaped by a theorem about what no tool can do, which is the pattern Module 12 generalises.

3 What a stack cannot do

A STACK COUNTS ONE THING AT A TIME

$\{a^n b^n c^n\}$ is **not context-free**. The stack can match the a's against the b's, but doing so empties it, and it then has nothing left with which to match the c's. **One matched pair is what a stack affords.**

Nor is $\{ww : w \text{ over } \{a,b\}\}$ context-free — a stack reverses whatever it stores, since the last pushed is the first popped, so it can recognise ww^R (a palindrome) and not ww (a repetition). Which is a memorable illustration of what the *discipline* of the memory costs, independently of its size.

The pumping lemma for context-free languages is the formal tool: it pumps *two* segments simultaneously, arising from a non-terminal that repeats along a path in the parse tree — which is Module 02's pigeonhole argument applied to tree depth rather than to string position.

And these are exactly the cases that arise in practice: declare-before-use, type agreement between a declaration and a use, and matching the number of arguments to the number of parameters are all beyond context-free — which is precisely why a compiler has a separate semantic analysis phase (CSCE 605 Module 06) rather than encoding everything in the grammar. The phase structure of a compiler is this hierarchy.

- Closed under union, concatenation, and Kleene star — combine the grammars with a new start symbol, or concatenate the right-hand sides.
- NOT closed under intersection. $a^n b^n c^*$ and $a^* b^n c^n$ are each context-free, and their intersection is $a^n b^n c^n$, which §3 just showed is not. A one-line proof of non-closure, and a useful one.
- NOT closed under complement, which follows from the above together with closure under union.
- Closed under intersection with a *regular* language — which preserves Module 02 §3's best proof technique for this class too, and is why that technique is worth the investment.
- And the deterministic context-free languages are a strict subclass of the context-free languages. So unlike the regular case, here non-determinism genuinely buys power — which is worth noting carefully, because it shows that whether non-determinism matters depends on the model. That is exactly why P versus NP is hard: the answer is no for finite automata, yes for pushdown automata, and unknown for polynomial-time Turing machines, so no general principle settles it.

4 The classes parsers actually use

Class	Parsed by	Property
LL(k)	Recursive descent with k symbols of lookahead; predictive, no backtracking.	Top-down, and left recursion must be eliminated first — which is why hand-written parsers restructure their grammars.
LR(k), LALR(1), SLR(1)	Shift-reduce parsing driven by a state table.	Strictly more powerful than LL, and what the yacc family generates. Handles left recursion naturally.

Class	Parsed by	Property
Deterministic context-free	A deterministic PDA.	Exactly the LR(1) languages — a theoretical class and a practical tool coinciding precisely, which is satisfying and not accidental.
General context-free	Earley, CYK, or GLR.	$O(n^3)$ in general, and $O(n^{2.8})$ by reduction to matrix multiplication — which is a surprising connection and not a practical one.
Parsing expression grammars (PEG)	Packrat parsing with ordered choice.	NOT a subclass of the context-free languages — the two classes are incomparable. PEGs can express some non-CF languages and cannot express some CF ones, which students routinely assume otherwise.

WHY REAL LANGUAGES ARE NOT QUITE CONTEXT-FREE, AND IT DOES NOT MATTER

No real programming language is context-free. Declare-before-use, type agreement, arity checking, and definite assignment are all beyond the class (§3) — and attempts to push them into the grammar produce grammars nobody can read.

So the standard architecture splits the work by what each phase needs: a regular lexer, a context-free parser, and a separate semantic analysis phase for everything else (CSCE 605 Modules 02 through 06).

And each phase uses the weakest machinery that suffices — which gives **linear-time lexing, near-linear parsing, and a general-purpose checker only where one is genuinely needed**, rather than running the most powerful tool over everything.

That layering is the practical lesson of this module: use the weakest model that expresses your problem, because weaker models have better algorithms, better error messages, better tooling, and decidable questions. **It is the same argument as CSCE 625 Module 01's cheapest-sufficient-agent and CSCE 669 Module 02's prefer-convexity** — three courses arriving at one principle about choosing the least powerful adequate formalism.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — chapter 2	Grammars, pushdown automata, their equivalence, and the context-free pumping lemma — the reference for §1–§3.
Hopcroft, Motwani & Ullman — chapters 5–7	Stronger on the parsing classes of §4 , including the LR(1) characterisation.

Resource	What it covers for this module
Ford — Parsing Expression Grammars (free)	§4's PEG row , including why the class is incomparable with the context-free languages.
Grune & Jacobs — Parsing Techniques, 2nd edition (first edition free)	The comprehensive survey of §4 , with the first edition freely available from the author.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a grammar for balanced brackets** and derive three strings.
- 2 **Implement a pushdown automaton** for the same language and trace it.
- 3 **Write the ambiguous expression grammar** and exhibit both parse trees for one string.
- 4 **Stratify it by precedence** and confirm the ambiguity is gone.
- 5 **Run an ambiguous grammar through a parser generator** and read the conflict report.
- 6 **Prove $a^n b^n c^n$ not context-free** by the context-free pumping lemma.
- 7 **Prove it again by the intersection non-closure argument**, and compare.
- 8 **Show that a PDA can recognise ww^R and argue it cannot recognise ww .**
- 9 **Find a PEG that accepts a non-context-free language.**
- 10 **For a language you use, identify which checks are regular, which context-free, and which neither.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Relate grammars to pushdown automata and say why both names are used.
- 2 Why is a stack exactly what matched counting needs?
- 3 Give the five-level hierarchy and the memory each has.
- 4 Why does parsing care about ambiguity more than recognition does?
- 5 Why do parser generators report conflicts rather than verdicts?
- 6 Why is $a^n b^n c^n$ beyond a stack? Why is ww ?
- 7 Give the one-line proof of non-closure under intersection.
- 8 Does non-determinism buy power here? What does that imply about P vs NP?
- 9 Name five parsing classes, and which coincides with a theoretical class.

10 Why is no real language context-free, and what architecture follows?